



Spec Kit 小白指南

GitHub 官方规格驱动开发 (Spec-Driven Development) 工具包 —— 先写规格书，再让 AI 照章施工

139.8K

GitHub Stars

specify

官方 CLI (Python)

25+

兼容 AI 编码代理

MIT

完全开源

Q1

这是什么？一句话能说清吗？

Spec Kit 是 GitHub 官方开源的“规格驱动开发”工具包：强制 AI 编码代理在写代码之前，先产出一份结构化规格书，再按规格一步步施工。它由三部分组成：

- **specify CLI** (Python 写的命令行工具)：一条命令 `specify init` 就给项目装好全套规格模板和代理技能；
- 一套 **/speckit-* 技能 (Skill)**：constitution、specify、plan、tasks、implement、converge 等 10 个，装进 Claude Code / Copilot / Cursor 等代理里当“流程指挥棒”；
- 一组**确定性脚本**：PowerShell / Bash 脚本负责建分支、建目录、填模板编号这类机械活，不让 AI 碰运气。

✦ 推文说的 `/constitution → /specify → /clarify → /plan → /tasks → /implement` 就是这套流程；截至本文实测 (v1.1.x)，命令已升级为 `/speckit-*` 技能形态，并新增了 converge (收敛对账) 环节。

Q2

"vibe coding 的最大问题"到底是什么？

推文开头说 GitHub "解决了 vibe coding 的最大问题"。这个问题就是：**对着 AI 随口丢模糊指令，然后祈祷它别把项目搞砸**。具体表现为三种翻车：

- **理解偏差**：你说"做个相册功能"，AI 脑补的可能是 5 种不同的东西；
- **上下文漂移**：聊到第 20 轮，AI 早就忘了第 2 轮的约定，改 A 破 B；
- **不可验收**：没有白纸黑字的标准，你说没做好，它说自己做好了，扯不清。

Spec Kit 的解法用一句话说：**先逼 AI 把"要建什么"写成可验收的规格书，问题在动工前暴露，而不是在代码里烂掉**。这就是本知识库已学的 **规格书先行 (Spec-Driven)** 思想的官方工业版。

Q3

它是怎么做到的？——"权力反转"

仓库里的 `spec-driven.md` 有一段纲领性论述，叫 **The Power Inversion (权力反转)**：几十年来代码是王，规格书是脚手架——写完代码就扔。AI 时代反过来了：**规格书是源头 (source of truth)**，代码只是规格书在某种语言/框架下的"表达产物"。改需求 = 改规格再重新生成；修 bug = 修规格里的漏洞。

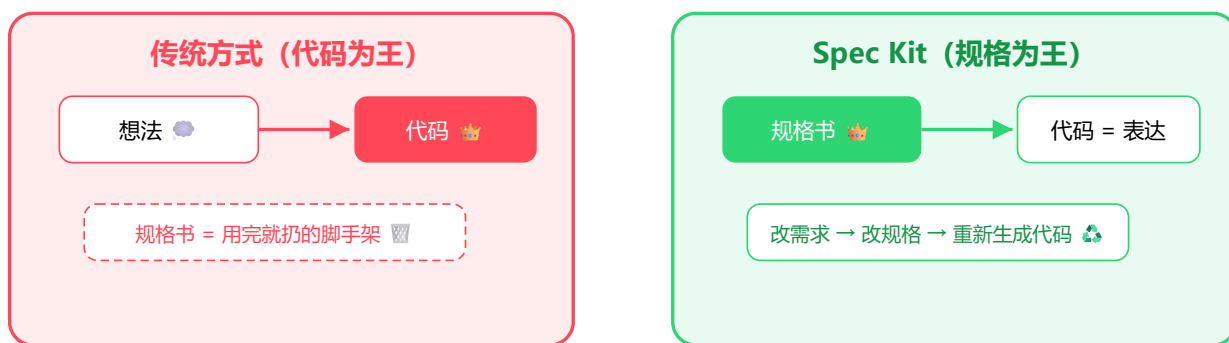


图1：权力反转——规格书从"脚手架"升级为"源头"

🔥 类比已学概念：这正是知识库里的 [[规格书先行SpecDriven]] (ai-website-cloner 的"侦察→合同→施工") 的官方放大版——官方直接把"主观感受不可复现，客观规格才能当合同"写成了产品。

Q4

七个 /speckit-* 技能各干什么？

可选质检：analyze（交叉一致性）· checklist（需求检查单）

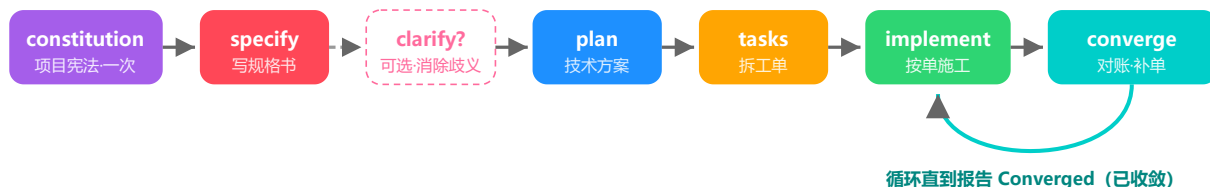


图2：SDD 主流程——宪法一次，specify→plan→tasks 每个功能一轮，implement↔converge 循环收敛

- **constitution**：定项目“根本法”（质量原则、技术红线），**全项目只写一次，之后每个功能的规格都得守它**；
- **specify**：把一句自然语言需求变成结构化规格书（用户故事按 P1/P2/P3 排优先级、每条带 Given/When/Then 验收场景）；
- **clarify**（可选）：施工前让 AI 主动提问，把含糊处问死；
- **plan**：定技术栈与架构决策（例：Vite + SQLite，图片留本地）；
- **tasks**：把规格+方案拆成有依赖顺序的任务清单（思想同已学的 [[工单任务图与前沿调度]]）；
- **implement**：按单施工；
- **converge**：**拿代码和 spec/plan/tasks 逐项对账，把没建成的部分追加为新任务，循环施工直到报告 "Converged"**——这是 2026 新增的环节，防“AI 说做完了其实没做完”。

Q5

specify init 到底往项目里装了什么？（实测解剖）

本指南的实验真实跑了一次 `specify init demo-todo --integration claude --non-interactive`，生成物如下：

```
demo-todo/
├── .specify/                                ← Spec Kit 的"项目舱"
│   ├── memory/constitution.md              ← 项目宪法（占位模板，等 /speckit-constitution 填）
│   ├── templates/                         ← spec / plan / tasks / checklist 四大模板
│   ├── scripts/powershell/                ← 确定性脚本：建 feature 目录、查前置条件...
│   ├── workflows/speckit/workflow.yml      ← 带人工 review 门的工作流定义
│   └── specs/001-photo-albums/spec.md      ← （实验手填）真正的规格书放这里
├── .claude/skills/speckit-*/              ← 装进 Claude Code 的 10 个 SKILL.md
├── speckit-constitution / -specify / -clarify / -plan / -tasks
└── speckit-implement / -converge / -analyze / -checklist / -taskstoissues
```

✦ 两个设计亮点：① 模板里全是结构槽位——spec 模板强制"用户故事可独立测试、P1 单独拿出来也是能用的 MVP"；② 技能 (SKILL.md) + 脚本分工——AI 负责判断和写内容，建目录/编号这类机械活交脚本，思想同已学的 [[AgentSkills技能包]] 与 [[源码即Prompt]]。

Q6

为什么"宪法"要先写？它约束谁？

`constitution.md` 是项目的"根本法"：列出核心原则（如 Library-First、Test-First 不可妥协、可观测性），**之后所有 spec、plan、tasks 生成时都要引用它做合规检查，冲突时宪法赢**。比如宪法写了"测试先行不可谈判"，AI 就不许在 tasks 里把"补测试"排到最后。这和知识库已学的 [[提示词权威边界]]（宪法管审美、军规管物流）是同一个思想在不同层级的落点：

	提示词权威边界 (xxd_panels)	Spec Kit 宪法 (项目级)
管什么	一次生成里的风格与结构产权	整个项目所有功能的质量红线
写几次	每份提示词都带	每项目一次，跨功能复用
冲突时	宪法（美学原文）逐字优先	宪法原则压过单点需求

Q7

它和已学项目是什么关系？（对比定位）

项目	做了什么	与 Spec Kit 的关系
ai-website-cloner	组件级"测量→spec→施工"	同思想的小型自造版；Spec Kit 是 GitHub 官方全流程版
six_skills (implement-spec)	spec 拆成 DAG 工单并行施工	Spec Kit 的 tasks 环节做同伴事（线性清单为主）；converge 补上其对账短板
archify (Agent Skill)	SKILL.md + CLI 门禁→单文件产物	同为"技能+CLI 门禁"形态，Spec Kit 的产物是整套项目而非单 HTML
teamai-cli	团队 AI 资产 Git 分发	Spec Kit 生成的 .specify/ 恰是可提交、可评审、可分发的团队资产

📌 一句话定位：Spec Kit = 把知识库里散落的"规格先行 + 工单拆解 + 技能化 + 确定性脚本门禁"四件套，用官方身份打包成一个开箱即用产品。

Q8

除了造新功能，它还能干什么？

v1.x 把三条流程做成了**独立入口**（不是必须三连）：

- **SDD 造功能**（核心，随 init 自带）：`specify → plan → tasks → implement → converge`；
- **Bug 修复**（扩展，`specify extension add bug`）：`assess → fix → test`，诊断、修复、验证三段分离，产出报告到 `.specify/bugs/<slug>/`，结论分 `verified / partial / failed`——**没验证就不算修好**；
- **点子评估**（扩展，`specify extension add assess`）：`intake → research → define → shape → decide`，不需要任何代码就能跑，最终给出 `go / needs-clarification / kill` 决定。

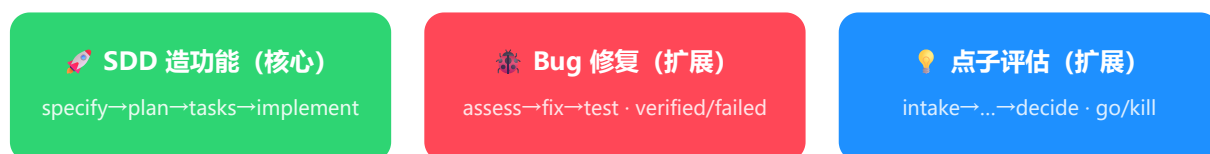


图3：三条独立流程入口——扩展用 `specify extension add` 按需安装



这个项目给我什么？（能力清单 + 我能用来干什么）

- ☒ 一条命令给任意项目装上 SDD 骨架（模板+脚本+代理技能）；
- ☒ 让 Claude Code / Copilot 等 25+ 代理按同一套纪律施工；
- ☒ 规格书、计划、任务全部落盘为 Markdown——可提交、可评审、可 diff；
- ☒ `converge` 对账循环：给"AI 宣称完工"上一个验收锁；
- ☒ `bug / assess` 扩展：修 bug 和评点子也有流程，不只造新功能。

结合我的水平，最实际的 4 个用途：

- ① 本仓库 400+ 子项目的练习项目，一律用 `specify init` 起步，练规格书写法；
- ② 借它的 spec 模板反哺知识库的 [[规格书先行SpecDriven]] 实践；
- ③ 给团队/个人 Claude 环境装 `/speckit-*` 技能，统一 AI 施工纪律（思路接 [[团队Harness的Git原生分发]]）；
- ④ 用 `assess` 扩展评估新点子，避免一上来就写代码。



动手实验：5 步跑通 Spec Kit（全部命令实测）

1 安装 specify CLI

```
pip install F:/reminder/speckit/repo # 从本地克隆安装（也可 uv tool install specify-cli）
specify --version
```

目的：装上官方 CLI。**实现了：**本机, Python 3.14 直接装好。

✅ 预期结果：`specify 1.1.1.dev0`（实测输出）

2 一键生成 SDD 项目骨架

```
specify init demo-todo --integration claude --non-interactive
```

目的：体验"一条命令装好全套"。**实现了：**生成 .specify/ 项目舱 + 10 个 .claude/skills 技能。

✅ 预期结果：终端打出 Next Steps 面板，列出 /speckit-constitution → -specify → -plan → -tasks → -implement → -converge 六连（实测）

⚠️ 坑：老教程里的 `--no-git` 选项已不存在（实测报 No such option）；仓库最新版也不再叫 /speckit.constitute 这类旧命令名。

3 环境体检：specify check

```
specify check
```

目的：看它支持哪些代理。本机装了哪个。**实现了：**逐个探测 30+ 代理 CLI。

✅ 预期结果：一长串 • / ○ 探测列表 + `Specify CLI is ready to use!`（实测）

4

亲手写第一份规格书

```
mkdir specs/001-photo-albums
cp .specify/templates/spec-template.md specs/001-photo-albums/spec.md
# 然后按模板填: 3 条用户故事 (P1/P2/P3), 每条带 Given/When/Then 验收场景
```

目的: 体会"规格书是合同"——故事必须可独立测试。 **实现了:** 照片整理器规格 (按月分组 P1 / 瓷砖预览 P2 / 元数据入 SQLite P3)。

✅ 预期结果: `specs/001-photo-albums/spec.md` 状态 Draft、三条故事各自可验收 (实验已产出 59 行直实规格)

⚠️ 说明: 官方脚本 `create-new-feature.ps1` 在本机被安全策略拦截 (外部代码), 此处用等价手动法达成同一产物——这正是"确定性脚本可被人工替代"的证明。

5

看扩展机制 + 启动代理真跑流程

```
specify extension list          # 实测输出: No extensions installed
cd demo-todo && claude          # 在项目里启动 Claude Code, 逐个调用 /speckit-
* 技能
```

目的: 确认扩展按需安装、技能进代理后即用。 **实现了:** 技能文件已就位 (SKILL.md 带 frontmatter 与钩子检查逻辑)。

✅ 预期结果: 代理里输入 / 能看到 speckit 全家桶: 完整 SDD 六连跑完得到 Converged 报告 (本实验止步于结构验证, 六连属重活留给读者)

 **知识关联:** 本指南概念已入 Obsidian 知识库——新概念 [\[\[项目宪法Constitution\]\]](#)、[\[\[规格收敛循环Converge\]\]](#); 互链 [\[\[规格书先行SpecDriven\]\]](#) (本次补记 spec-kit 来源与"权力反转")、[\[\[工单任务图与前沿调度\]\]](#)、[\[\[提示词权威边界\]\]](#)、[\[\[AgentSkills技能包\]\]](#)。

 2026-10-03 · 学习自 x.com/txbrraa 推文 → [github/spec-kit](https://github.com/spec-kit) (139.8★ · MIT · v1.1.x) · 实验环境 Windows 11 + Python 3.14